WNDRVR
SUPPORT NETWORK

API REFERENCE: APPLICATION SECURITY, SR0650

Copyright Notice

Copyright © 2022 Wind River Systems, Inc.

All rights reserved. No part of this publication may be reproduced or transmitted in any form or by any means without the prior written permission of Wind River Systems, Inc.

Wind River, Simics, Tornado, and VxWorks are registered trademarks of Wind River Systems, Inc. Helix, Pulsar, Rocket, Titanium Cloud, Titanium Control, Titanium Core, Titanium Edge, Titanium Edge SX, Titanium Server, and the Wind River logo are trademarks of Wind River Systems, Inc. Any third-party trademarks referenced are the property of their respective owners. For further information regarding Wind River trademarks, please see:

www.windriver.com/company/terms/trademark.html

This product may include software licensed to Wind River by third parties. Relevant notices (if any) are provided for your product on the Wind River download and installation portals:

https://delivers.windriver.com/

https://windshare.usa.windriver.com/

Wind River may refer to third-party documentation by listing publications or providing links to third-party websites for informational purposes. Wind River accepts no responsibility for the information provided in such third-party documentation.

Corporate Headquarters

Wind River
500 Wind River Way
Alameda, CA 94501-1153
U.S.A.
Toll free (U.S.A.): +1-800-545-WIND
Telephone: +1-510-748-4100
Facsimile: +1-510-749-2010

For additional contact information, see the Wind River website:

www.windriver.com

For information on how to contact Customer Support, see:

www.windriver.com/support

API Reference: Application Security, SR0650

20 July 2020

1. LOGINLIB

Name

loginLib - user login/password subroutine library

Layer

USER_MANAGEMENT

Routines

loginUserVerify( ) - verify a user name and password in the login table

Description

This library provides a routine to validate a login name/password pair. The login name/password pair is looked up in the VxWorks user database managed by the kernel's user management feature.

Support for the VxWorks user management feature is enabled in a VxWorks Source Build project via the **USER_MANAGEMENT** layer. The VxWorks user database support as well as user identification support is included in the system via the **INCLUDE_USER_IDENTIFICATION** component

Include files

loginLib.h

See also

Name

loginUserVerify( ) - verify a user name and password in the login table

Layer

USER_MANAGEMENT

Synopsis

```
STATUS loginUserVerify
    (
    char * name,    /* name of user */
    char * passwd   /* password of user */
    )
```

Description

This routine verifies a user entry in the kernel login table.

Returns

OK, or **ERROR** if the user name or password is not found.

Errno

Possible errnos set by this routine include:

EINVAL

An invalid argument is passed to the routine.

S_loginLib_UNKNOWN_USER

Unknown user name name.

S_loginLib_INVALID_PASSWORD

Invalid password passwd for name

See also

loginLib

2. USERIDENTLIB

Name

userIdentLib - VxWorks user identification library

Layer

USER_MANAGEMENT

Routines

userAuthenticate( ) - authenticate a user by name and password (system call) **userPasswordUpdate( )** - update the password for an existing user (system call) **_getpwuid( )** - search the user database for a user ID (system call) **_getpwnam( )** - search the user database for a user name (system call) **getpwuid( )** - search the user database for a user ID (POSIX) **getpwnam( )** - search the user database for a user name (POSIX) **getpwuid_r( )** - search the user database for a user ID (POSIX) **getpwnam_r( )** - search the user database for a user ID (POSIX)

Description

This library provides the facility needed to authenticate a VxWorks user as well as update a user's password from the RTP environment.

This library also provides the POSIX **getpwuid( )**, **getpwnam( )**, **getpwuid_r( )** and **getpwnam_r( )** API. Note that the **getpwuid( )** and **getpwnam( )** API are not thread-safe.

Random number generation

For random numbers of good quality the IPNET Entropy Gathering Daemon (EGD) is used when the OPENSSL feature is enabled in the VxWorks Source Build. If not, the regular VxWorks random number generator is used instead.

Configuration

The user identification feature is included when the **INCLUDE_USER_IDENTIFICATION** component is added to a vxWorks image project (VIP). Note that, for convenience, including the **INCLUDE_SECURITY** component will automatically add the user identification feature and all its dependencies.

Include files

none

Name

userAuthenticate( ) - authenticate a user by name and password (system call)

Layer

USER_MANAGEMENT

Synopsis

```
STATUS userAuthenticate
    (
```

```
    const char * loginName,     /* login name for the user to authenticate */
    const char * clearPassword, /* cleartext passwd of user to authenticate */
    uid_t *      pUserId,       /* where to store the user's user ID */
    gid_t *      pGroupId       /* RESERVED */
    )
```

This routine authenticates a user given a name and password. If the user is known to the system and the pUserId parameter is not **NULL**, the user's user ID is provided to the caller. Note: the pGroupId parameter is currently unused and reserved.

Description

none

Returns

OK, or **ERROR** if the user is not found by name or the provided password does not match the record.

Errno

S_userIdentLib_AUTHENTICATION_FAILED

The user could not be authenticated: either this user does not exist or the provided password does not match the record.

See also

userIdentLib

Name

userPasswordUpdate( ) - update the password for an existing user (system call)

Layer

USER_MANAGEMENT

Synopsis

```
STATUS userPasswordUpdate
    (
    const char * loginName,        /* login name of user to update */
    const char * oldClearPassword, /* user's old password in cleartext */
    const char * newClearPassword, /* user's new password in cleartext */
    )
```

This routine updates the password of the user loginName. The new password is to be passed as cleartext in newClearPassword. No privilege verifications are performed by this routine but the old password must be passed and must be successfully verified before the password is changed. The old password is to be passed as cleartext in oldClearPassword.

Description

none

Returns

OK, or **ERROR** if the password could not be updated.

Errno

S_userIdentLib_AUTHENTICATION_FAILED

The user could not be authenticated: either this user does not exist or the provided password does not match the record.

See also

userIdentLib

Name

_getpwuid( ) - search the user database for a user ID (system call)

Layer

USER_MANAGEMENT

Synopsis

```
int _getpwuid
    (
    int    userId,            /* UID of user whose info is of interest */
    struct passwd * pwd,      /* where to store the user information */
    char * buffer,            /* allocated memory for information */
    size_t bufsize            /* size of allocated memory */
    )

This routine supports the getpwuid() and getpwuid_r() functions. See the
manuals of those routines for a detailed description of the parameters.

RETURNS: zero if successful, otherwise an error code indicative of the
error.

ERRNO:
\is
\i EINVAL
One of the <pwd> or <result> parameters is null.
\i ENOMEM
Not enough memory left to allocate internal data structures.
\i ERANGE
The buffer provided for storing the user information is too small.
\i ESRCH
No matching user database entry was found for the provided UID.
\ie
```

Description

none

Returns

Not Available

Errno

Not Available

See also

userIdentLib

Name

_getpwnam( ) - search the user database for a user name (system call)

Layer

USER_MANAGEMENT

Synopsis

```
int _getpwnam
    (
    const char * loginName,   /* name of user whose info is of interest */
    struct passwd * pwd,      /* where to store the user information */
    char * buffer,            /* allocated memory for information */
    size_t bufsize            /* size of allocated memory */
    )

This routine supports the getpwnam() and getpwnam_r() functions. See the
manuals of those routines for a detailed description of the parameters.

RETURNS: zero if successful, otherwise an error code indicative of the
error.

ERRNO:
\is
\i EINVAL
One of the <loginName>, <pwd> or <result> parameters is null.
\i ENOMEM
Not enough memory left to allocate internal data structures.
\i ERANGE
The buffer provided for storing the user information is too small.
\i ESRCH
No matching user database entry was found for the provided UID.
\ie
```

Description

none

Returns

Not Available

Errno

Not Available

See also

userIdentLib

Name

getpwuid( ) - search the user database for a user ID (POSIX)

Layer

USER_MANAGEMENT

Synopsis

```
struct passwd * getpwuid
    (
    uid_t userId   /* UID of user whose information is of interest */
    )
```

Description

This routine searches the user database for entries matching the provided user ID.

Note that this routine is not thread-safe: the **passwd** data structure, and all associated storage areas, pointed to by the pointer returned by this function will be overwritten by subsequent calls to **getpwuid( )**. See **getpwuid_r( )** for the thread-safe version.

Returns

a pointer to a static **passwd** structure if a matching entry is found, a **NULL** pointer otherwise or if an error occurred.

Errno

Not Available

See also

userIdentLib

Name

getpwnam( ) - search the user database for a user name (POSIX)

Layer

USER_MANAGEMENT

Synopsis

```
struct passwd * getpwnam
    (
    const char * loginName   /* name of user whose information is of interest */
    )
```

Description

This routine searches the user database for entries matching the provided user name.

Note that this routine is not thread-safe: the **passwd** data structure, and all associated storage areas, pointed to by the pointer returned by this function will be overwritten by subsequent calls to **getpwnam( )**. See **getpwnam_r( )** for the thread-safe version.

Returns

a pointer to a static **passwd** structure if a matching entry is found, a **NULL** pointer otherwise or if an error occurred.

Errno

Not Available

See also

userIdentLib

Name

getpwuid_r( ) - search the user database for a user ID (POSIX)

Layer

USER_MANAGEMENT

Synopsis

```
int getpwuid_r
    (
    uid_t            userId,    /* UID of user whose information is of interest */
    struct passwd *  pwd,       /* where to store the user information */
    char *           buffer,    /* allocated memory for information */
    size_t           bufsize,   /* size of allocated memory */
    struct passwd ** result     /* where to store the address of the user info */
    )
```

Description

This routine searches the user database for entries matching the provided user ID. If the search is successful, the data structure pointed to by the pwd parameter holds the relevant user information and the address pointed to by the result parameter is set with the address of the pwd structure. If the search is not successful the data structure pointed to by the pwd parameter is not updated and the address pointed to by the result parameter is set to **NULL**.

The buffer parameter should point to a block of allocated memory of at least **MAX_LOGIN_NAME_LEN** in size.

Returns

zero if successful, otherwise an error code indicative of the error.

Errnos

EINVAL

One of the pwd or result parameters is null.

ENOMEM

Not enough memory left to allocate internal data structures.

ERANGE

The buffer provided for storing the user information is too small.

ESRCH

No matching user database entry was found for the provided UID.

See also

userIdentLib

Name

getpwnam_r( ) - search the user database for a user ID (POSIX)

Layer

USER_MANAGEMENT

Synopsis

```
int getpwnam_r
    (
    const char *     loginName,   /* name of user whose info is of interest */
    struct passwd *  pwd,         /* where to store the user information */
    char *           buffer,      /* allocated memory for information */
    size_t           bufsize,     /* size of allocated memory */
    struct passwd ** result       /* where to store the address of the user info */
    )
```

Description

This routine searches the user database for entries matching the provided user name. If the search is successful, the data structure pointed to by the pwd parameter holds the relevant user information and the address pointed to by the result parameter is set with the address of the pwd structure. If the search is not successful the data structure pointed to by the pwd parameter is not updated and the address pointed to by the result parameter is set to **NULL**.

The buffer parameter should point to a block of allocated memory of at least **MAX_LOGIN_NAME_LEN** in size.

Returns

zero if successful, otherwise an error code indicative of the error.

Errnos

EINVAL

One of the loginName, pwd or result parameters is null.

ENOMEM

Not enough memory left to allocate internal data structures.

ERANGE

The buffer provided for storing the user information is too small.

ESRCH

No matching user database entry was found for the provided UID.

See also

userIdentLib